VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Anagha D K(1BM23CS032)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
August-2025 to December-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Artificial Intelligence (23CS5PCAIN)" carried out by **Anagha D K(1BM23CS032),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Prof.Swathi Sridharan Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Github Link: **https://github.com/anaghadk/AI-lab**

Infosys
Navigate your next

COURSE COMPLETION CERTIFICATE

The certificate is awarded to

Anagha D K

for successfully completing the course

Introduction to Artificial Intelligence

on November 25, 2025

Infosys | Springboard

Congratulations! You make us proud!

Satheesha B. N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Tuesday, November 25, 2025
To verify, scan the QR code at https://verify.onwingspan.com

Infosys
Navigate your next

COURSE COMPLETION CERTIFICATE

The certificate is awarded to

Anagha D K

for successfully completing the course

Introduction to Deep Learning

on November 25, 2025

Infosys | Springboard

Congratulations! You make us proud!

Satheesha B. N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Tuesday, November 25, 2025
To verify, scan the QR code at https://verify.onwingspan.com

Infosys
Navigate your next

COURSE COMPLETION CERTIFICATE

The certificate is awarded to

Anagha D K

for successfully completing the course

Introduction to Natural Language Processing

on November 25, 2025

Infosys | Springboard

Congratulations! You make us proud!

Satheesha B. N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Tuesday, November 25, 2025
To verify, scan the QR code at https://verify.onwingspan.com

Infosys
Navigate your next

CERTIFICATE OF ACHIEVEMENT

The certificate is awarded to

Anagha D K

for successfully completing

Artificial Intelligence Foundation Certification

on November 25, 2025

Infosys | Springboard

Congratulations! You make us proud!

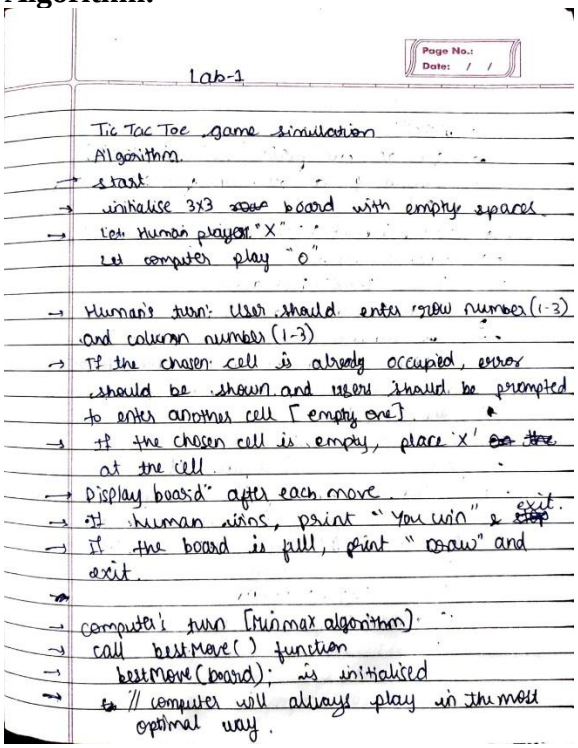
Satheesha B. N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

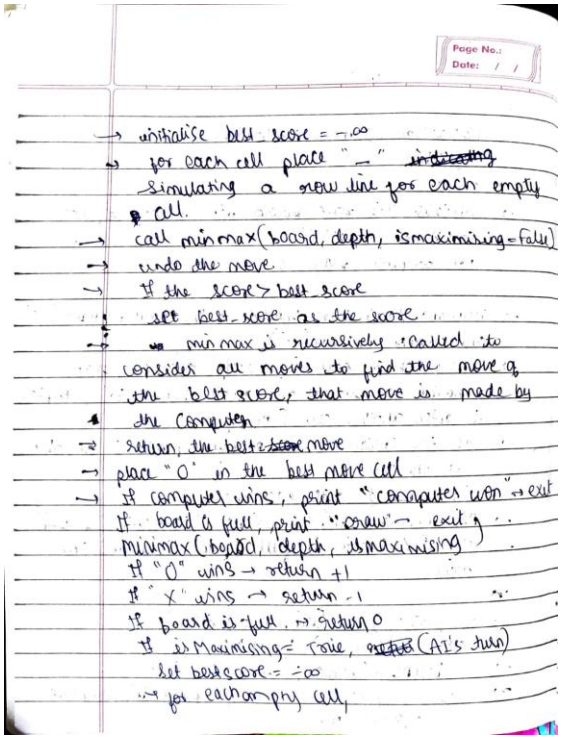
Issued on: Tuesday, November 25, 2025
To verify, scan the QR code at https://verify.onwingspan.com

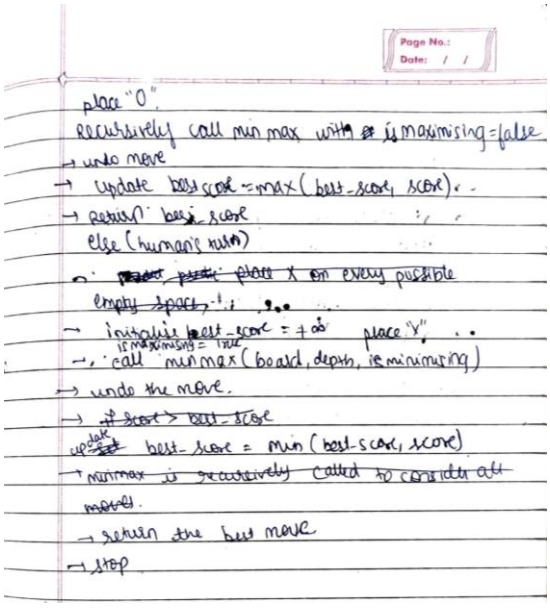
Program 1

Implement Tic –Tac –To Game

Algorithm:







Code:

```
from typing import List, Optional, Tuple
PLAYER, AI = 'O', 'X'
def print_board(b: List[str]) -> None:
```

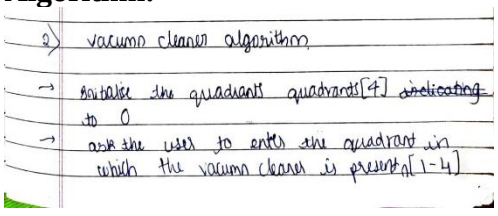
```
    print()
    for i in range(0, 9, 3):
        row = [b[i], b[i+1], b[i+2]]
        print(" | ".join(c if c != ' ' else str(i+j+1) for j, c in enumerate(row)))
        if i < 6: print("--+---+--")
    print()

def winner(b: List[str]) -> Optional[str]:
    lines = [
        (0,1,2),(3,4,5),(6,7,8), # rows
        (0,3,6),(1,4,7),(2,5,8), # cols
        (0,4,8),(2,4,6) # diags
    ]
    for a, c, d in lines:
        if b[a] != ' ' and b[a] == b[c] == b[d]:
            return b[a]
    return None

def full(b: List[str]) -> bool:
    return all(c != ' ' for c in b)

def score_terminal(b: List[str]) -> Optional[int]:
    w = winner(b)
    if w == AI: return 1
    if w == PLAYER: return -1
    if full(b): return 0
    return None # not terminal

def minimax(b: List[str], is_maximizing: bool, alpha: int, beta: int) -> Tuple[int, Optional[int]]:
    s = score_terminal(b)
    if s is not None:
        return s, None

    best_move = None
    if is_maximizing:
        best_score = -10
        for i in range(9):
            if b[i] == ' ':
                b[i] = AI
                val, _ = minimax(b, False, alpha, beta)
                b[i] = ' '
                if val > best_score:
                    best_score, best_move = val, i
                alpha = max(alpha, best_score)
                if beta <= alpha:
                    break
        return best_score, best_move
    else:
        best_score = 10
        for i in range(9):
            if b[i] == ' ':
                b[i] = PLAYER
                val, _ = minimax(b, True, alpha, beta)
                b[i] = ' '
                if val < best_score:
                    best_score, best_move = val, i
                beta = min(beta, best_score)
```

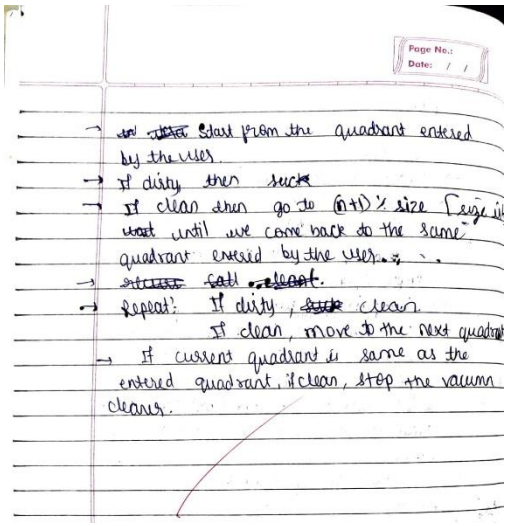
```
            if beta <= alpha:
                break
        return best_score, best_move

def ai_move(b: List[str]) -> int:
    # If first move, pick center if free for speed
    if b[4] == ' ':
        return 4
    _, move = minimax(b, True, -10, 10)
    return move if move is not None else next(i for i,c in enumerate(b) if c==' ')

def human_move(b: List[str]) -> int:
    while True:
        try:
            pos = int(input("Your move (1-9): ")) - 1
            if 0 <= pos < 9 and b[pos] == ' ':
                return pos
        except ValueError:
            pass
        print("Invalid move. Try again.")

def main():
    board = [' '] * 9
    print("You are O, AI is X. AI goes first.")
    print_board(board)

    turn = 'X' # AI first; set to 'O' if you want human to start
    while True:
        if turn == AI:
            move = ai_move(board)
            board[move] = AI
            print("AI moves to", move+1)
        else:
            move = human_move(board)
            board[move] = PLAYER

        print_board(board)
        w = winner(board)
        if w or full(board):
            if w == AI: print("AI wins!")
            elif w == PLAYER: print("You win!")
            else: print("It's a draw.")
            break
        turn = PLAYER if turn == AI else AI

if __name__ == "__main__":
    main()
```

```
========================================
You are O, AI is X. AI goes first.

1 | 2 | 3
--+---+--
4 | 5 | 6
--+---+--
7 | 8 | 9

AI moves to 5

1 | 2 | 3
--+---+--
4 | X | 6
--+---+--
7 | 8 | 9

Your move (1-9): 1

O | 2 | 3
--+---+--
4 | X | 6
--+---+--
7 | 8 | 9

AI moves to 2

O | X | 3
--+---+--
4 | X | 6
--+---+--
7 | 8 | 9

Your move (1-9): 8

O | X | 3
--+---+--
4 | X | 6
--+---+--
7 | O | 9

AI moves to 4

O | X | 3
--+---+--
X | X | 6
--+---+--
7 | O | 9
```

```
Your move (1-9): 8

O | X | 3
--+---+--
4 | X | 6
--+---+--
7 | O | 9

AI moves to 4

O | X | 3
--+---+--
X | X | 6
--+---+--
7 | O | 9

Your move (1-9): 6

O | X | 3
--+---+--
X | X | O
--+---+--
7 | O | 9

AI moves to 3

O | X | X
--+---+--
X | X | O
--+---+--
7 | O | 9

Your move (1-9): 7

O | X | X
--+---+--
X | X | O
--+---+--
O | O | 9

AI moves to 9

O | X | X
--+---+--
X | X | O
--+---+--
O | O | X

It's a draw.
```

Implement vacuum cleaner agent
Algorithm:





Code:

```
import random

# Define environment with 4 quadrants
environment = {
    "A": random.choice(["Clean", "Dirty"]),
    "B": random.choice(["Clean", "Dirty"]),
    "C": random.choice(["Clean", "Dirty"]),
    "D": random.choice(["Clean", "Dirty"])
}

# Vacuum starts at position A
current_position = "A"

# Possible moves between quadrants
moves = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}

def vacuum_agent(position):
    global environment
    if environment[position] == "Dirty":
        print(f"Vacuum at {position}: Found Dirty → Suck")
        environment[position] = "Clean"
    else:
        print(f"Vacuum at {position}: Already Clean")

    # Move to a new quadrant
    new_position = random.choice(moves[position])
    print(f"Moving from {position} → {new_position}\n")
    return new_position

# Run the agent for a few steps
print("Initial Environment:", environment, "\n")
for _ in range(10):
```

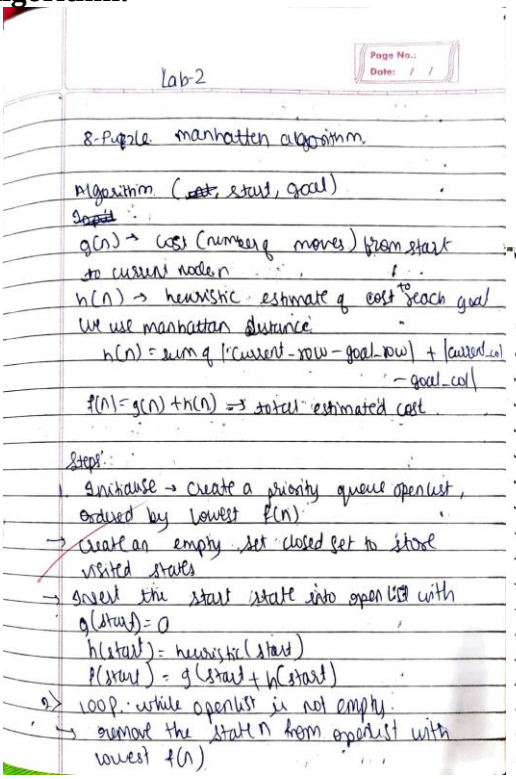
```
    current_position = vacuum_agent(current_position)

print("Final Environment:", environment)
```

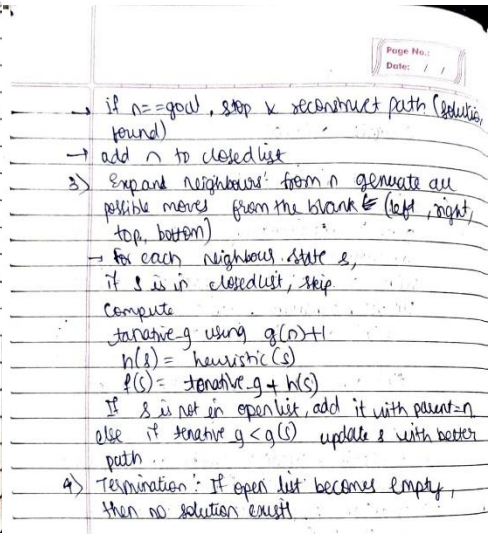
```
Initial Environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean', 'D': 'Dirty'}

Vacuum at A: Already Clean
Moving from A → C

Vacuum at C: Already Clean
Moving from C → D

Vacuum at D: Found Dirty → Suck
Moving from D → C

Vacuum at C: Already Clean
Moving from C → D

Vacuum at D: Already Clean
Moving from D → B

Vacuum at B: Already Clean
Moving from B → D

Vacuum at D: Already Clean
Moving from D → B

Vacuum at B: Already Clean
Moving from B → D

Vacuum at D: Already Clean
Moving from D → C

Vacuum at C: Already Clean
Moving from C → A

Final Environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean', 'D': 'Clean'}
```

Program 2

Implement 8 puzzle problems using Depth First Search (DFS)

Algorithm:





Code:

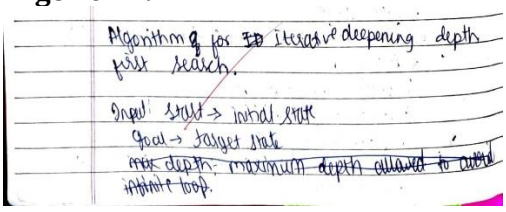
```
from collections import deque

GOAL = (1,2,3,4,5,6,7,8,0)

# Function to print the board nicely
def print_board(state):
    for i in range(0,9,3):
        print(state[i], state[i+1], state[i+2])
    print()

# Generate all legal neighbor states
def neighbors(state):
    i = state.index(0)  # blank position
    r, c = divmod(i, 3)
    for j in (i-3, i+3, i-1, i+1):
        if 0 <= j < 9:
            # prevent wrap-around in rows
            if (j == i-1 and c == 0) or (j == i+1 and c == 2):
                continue
            s = list(state)
            s[i], s[j] = s[j], s[i]
            yield tuple(s)

# Check if the puzzle is solvable
def is_solvable(state):
    arr = [x for x in state if x != 0]
    inv = 0
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] > arr[j]:
                inv += 1
    return inv % 2 == 0

# BFS to solve the puzzle
def bfs(start):
    if not is_solvable(start):
        return None
    queue = deque([start])
    parent = {start: None}  # keep track of moves
    while queue:
        cur = queue.popleft()
        if cur == GOAL:
            # reconstruct path
            path = []
            while cur is not None:
                path.append(cur)
                cur = parent[cur]
            return path[::-1]  # reverse to get start → goal
        for nb in neighbors(cur):
            if nb not in parent:
                parent[nb] = cur
                queue.append(nb)
    return None

# Main program
```

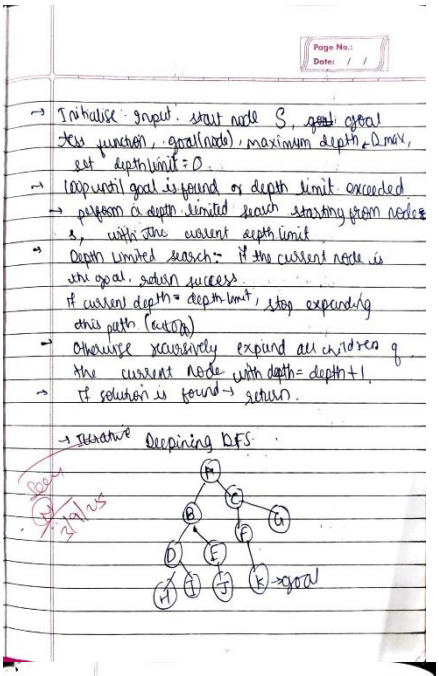
```
if __name__ == "__main__":
    print("Enter the 8-puzzle start state (0 for blank) row by row, separated by spaces:")
    user_input = []
    for i in range(3):
        row = input(f"Row {i+1}: ").split()
        if len(row) != 3:
            print("Please enter exactly 3 numbers per row.")
            exit()
        user_input.extend(int(x) for x in row)
    start = tuple(user_input)

    print("\nStart Board:")
    print_board(start)
    print("Solvable?", is_solvable(start))

    path = bfs(start)
    if path:
        print("Solved in", len(path)-1, "moves")
        for step, state in enumerate(path):
            print(f"Step {step}:")
            print_board(state)
    else:
        print("No solution (unsolvable)")
```

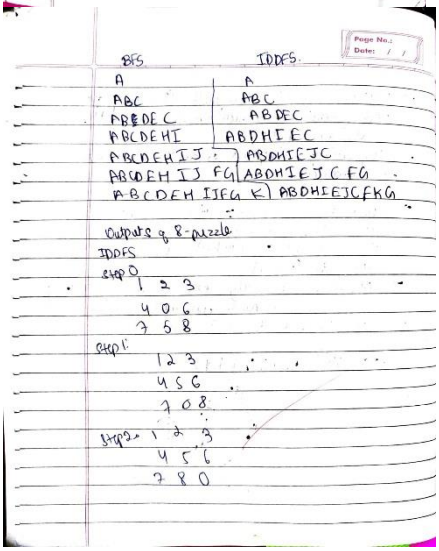
```
Enter the 8-puzzle start state (0 for blank) row by row, separated by spaces:
Row 1:  1 2 3
Row 2:  4 5 6
Row 3:  0 7 8

Start Board:
1 2 3
4 5 6
0 7 8

Solvable? True
Solved in 2 moves
Step 0:
1 2 3
4 5 6
0 7 8

Step 1:
1 2 3
4 5 6
7 0 8

Step 2:
1 2 3
4 5 6
7 8 0
```

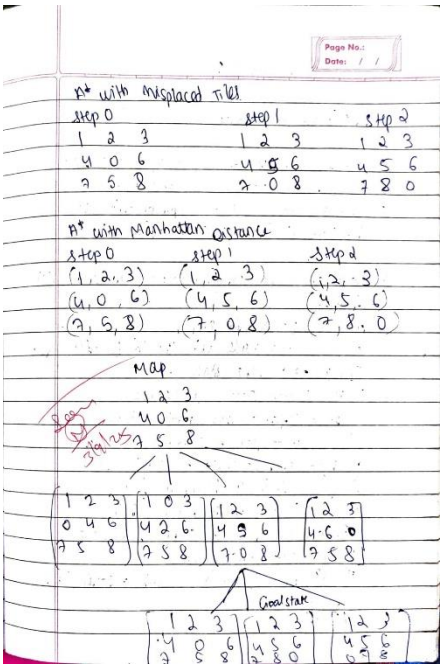
Implement Iterative deepening search algorithm

Algorithm:









Code:

```
from collections import deque

# Goal state
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Moves: up, down, left, right
MOVES = {
    'U': -3,
    'D':  3,
    'L': -1,
```

```
    'R':  1
}

def is_valid_move(pos, move):
    """Check if blank can move in given direction."""
    if move == 'U' and pos < 3: return False
    if move == 'D' and pos > 5: return False
    if move == 'L' and pos % 3 == 0: return False
    if move == 'R' and pos % 3 == 2: return False
    return True

def neighbors(state):
    """Generate valid neighbor states from current state."""
    new_states = []
    pos = state.index(0)  # blank position
    for move, offset in MOVES.items():
        if is_valid_move(pos, move):
            new_pos = pos + offset
            new_state = list(state)
            new_state[pos], new_state[new_pos] = new_state[new_pos], new_state[pos]
            new_states.append((tuple(new_state), move))
    return new_states

def dls(state, depth, visited, path):
    """Depth-limited DFS."""
    if state == GOAL:
        return path
    if depth == 0:
        return None
    visited.add(state)

    for neighbor, move in neighbors(state):
        if neighbor not in visited:
            result = dls(neighbor, depth - 1, visited, path + [move])
            if result is not None:
                return result
    return None

def iddfs(start, max_depth=50):
    """Iterative Deepening DFS."""
    for depth in range(max_depth):
        visited = set()
        path = dls(start, depth, visited, [])
        if path is not None:
            return path
    return None

# --- MAIN PROGRAM ---
if __name__ == "__main__":
    print("Enter the start state of the 8-puzzle (use 0 for blank):")
    nums = []
    for i in range(9):
        nums.append(int(input(f"Tile {i+1}: ")))
    start = tuple(nums)

    solution = iddfs(start, max_depth=30)
    if solution:
```

```
        print(f"\nSolution found in {len(solution)} moves:")
        print(" -> ".join(solution))
    else:
        print("No solution found within depth limit.")
```

```
In [8]: runfile('C:/Users/BMSCECSE/.spyder-py3/temp.py', wdir='C:/Users/BMSCECSE/.spyder-py3')
Enter the start state of the 8-puzzle (use 0 for blank):
Tile 1: 1
Tile 2: 2
Tile 3: 3
Tile 4: 4
Tile 5: 0
Tile 6: 6
Tile 7: 5
Tile 8: 8
Tile 9: 7

Solution found in 18 moves:
D -> L -> U -> U -> R -> D -> D -> R -> U -> L -> U -> L -> D -> D -> R -> U -> R -> D
```

(graph)

```
def dls(graph, current, goal, depth, path, visited, traversal):
    traversal.append(current)
    if depth == 0 and current == goal:
        return path + [current]
    if depth > 0:
        visited.add(current)
        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                result = dls(graph, neighbor, goal, depth - 1, path + [current], visited, traversal)
                if result is not None:
                    return result
        visited.remove(current)
    return None

def iddfs(graph, start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()
        traversal = []
        print(f"\nDepth Level {depth}:")
        path = dls(graph, start, goal, depth, [], visited, traversal)
        print("Traversal Order:", " -> ".join(traversal))
        if path:
            print("Goal found at this level!")
            return path
    return None

def main():
    print("Enter the number of nodes:")
    n = int(input())

    graph = {}
    print("Enter the nodes:")
    nodes = input().split()

    for node in nodes:
        graph[node] = []

    print("Enter the edges in the format 'A B' meaning edge from A to B (type 'done' to finish):")
```

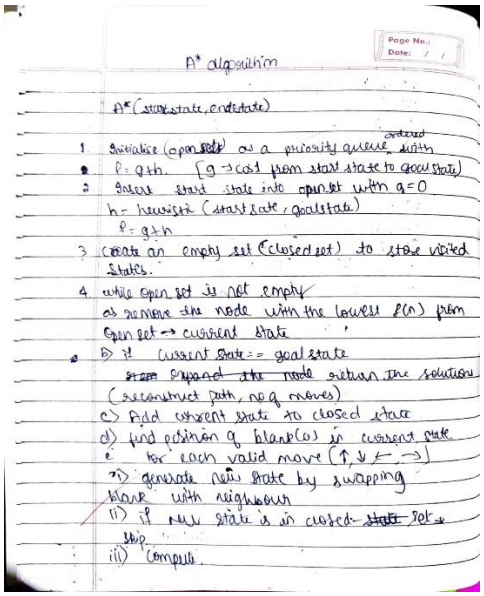
```
    while True:
        edge = input()
        if edge.lower() == 'done':
            break
        u, v = edge.split()
        if u in graph:
            graph[u].append(v)
        else:
            graph[u] = [v]

    start_node = input("Enter the start node: ")
    goal_node = input("Enter the goal node: ")
    max_depth = int(input("Enter the maximum depth limit: "))

    result = iddfs(graph, start_node, goal_node, max_depth)
    if result:
        print("\nGoal Path:", " -> ".join(result))
    else:
        print("\nGoal not found within depth limit.")

if __name__ == "__main__":
    main()
```

```
In [2]: runfile('C:/Users/BMSCECSE/untitled0.py', wdir='C:/Users/BMSCECSE')
Enter the number of nodes:
9
Enter the nodes:
a b c d e f g h i
Enter the edges in the format 'A B' meaning edge from A to B (type 'done' to finish):
a b
a c
b d
b e
c f
c g
d h
e i
done
Enter the start node: a
Enter the goal node: g
Enter the maximum depth limit: 3

Depth Level 0:
Traversal Order: a

Depth Level 1:
Traversal Order: a -> b -> c

Depth Level 2:
Traversal Order: a -> b -> d -> e -> c -> f -> g
Goal found at this level!

Goal Path: a -> c -> g
```

Program 3

Implement A* search algorithm

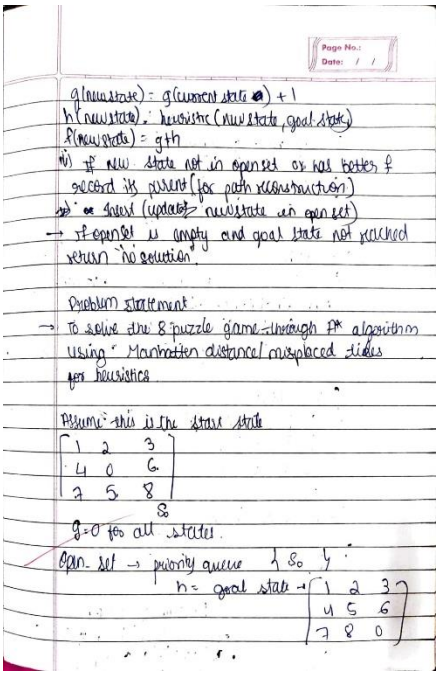
Algorithm:



A* algorithm

A*(start state, end state)

1. Initialize (open set) as a priority queue with
   f = g+h. [g → cost from start state to goal state]
2. Insert start state into open set with g=0
   h = heuristic (start state, goal state)
   f = g+h
3. create an empty set (closed set) to store visited states.
4. while open set is not empty
   a) remove the node with the lowest f(n) from open set → current state
   b) if current state == goal state
      ~~stop~~ expand the node return the solution (reconstruct path, no of moves)
   c) Add current state to closed state
   d) find position of blank(0) in current state
   e) for each valid move (↑, ↓, ←, →)
      i) generate new state by swapping blank with neighbours
      ii) if new state is in closed ~~state~~ set skip
      iii) Compute



g(new state) = g(current state) + 1
h(new state) = heuristic (new state, goal state)
f(new state) = g+h
i) if new state not in open set or has better f record its parent (for path reconstruction)
ii) ~~or~~ insert (update) new state in open set
→ if open set is empty and goal state not reached return 'no solution'.

Problem statement

→ To solve the 8 puzzle game through A* algorithm using "Manhatten distance/ misplaced tiles for heuristics

Assume this is the start state

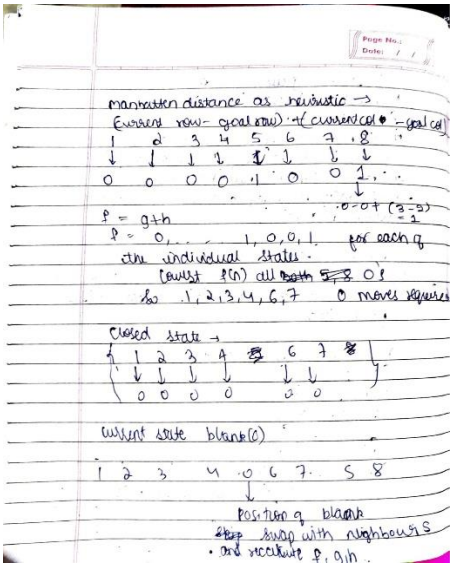
1	2	3
4	0	6
7	5	8

g=0 for all states.

Open set → priority queue { so }

h = goal state →

1	2	3
4	5	6
7	8	0



manhatten distance as heuristic →

Current row - goal row) + (current col - goal col)

1	2	3	4	5	6	7	8
↓	↓	↓	↓	↓	↓	↓	↓
0	0	0	0	1	0	0	1

f = g+h

f = 0, ... 1, 0, 0, 1, for each g the individual states.

lowest f(n) all ~~both~~ 0's

so 1, 2, 3, 4, 6, 7 moves required

Closed state →

1	2	3	4	~~5~~	6	7	~~8~~
0	0	0	0		0	0	

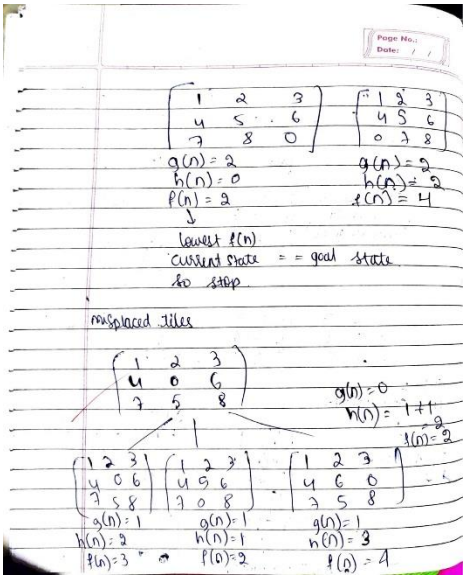
current state blank(6)

1 2 3 4 0 6 7 5 8

↓

Position of blank ~~stop~~ swap with neighbours

and recursive f, g, h.



1	2	3
4	5	6
7	8	0

g(n) = 2
h(n) = 0
f(n) = 2

1	2	3
4	5	6
0	7	8

g(n) = 2
h(n) = 2
f(n) = 4

↓

lowest f(n)
current state == goal state
so stop

misplaced tiles

1	2	3
4	0	6
7	5	8

g(n) = 0
h(n) = 1+1 = 2
f(n) = 2

1	2	3
4	0	6
7	5	8

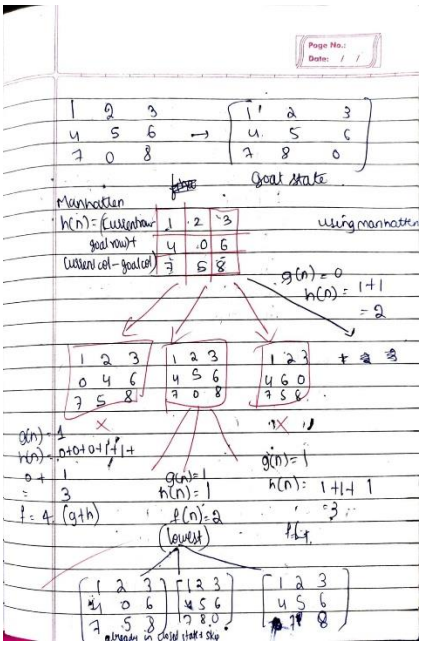
g(n) = 1
h(n) = 2
f(n) = 3

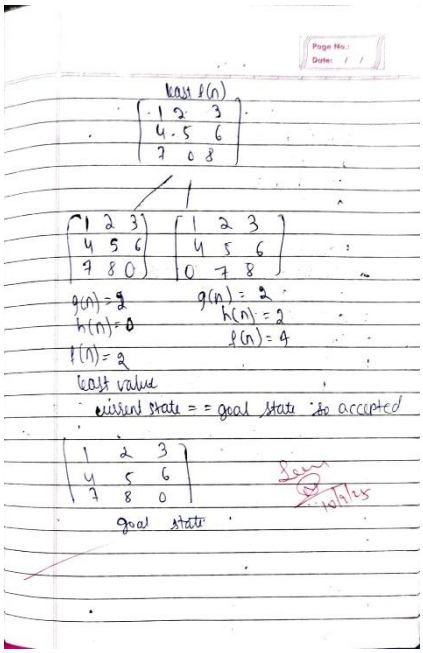
1	2	3
4	5	6
7	0	8

g(n) = 1
h(n) = 1
f(n) = 2

1	2	3
4	6	0
7	5	8

g(n) = 1
h(n) = 3
f(n) = 4





Code:

```
import heapq

# Goal state
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Moves: up, down, left, right
MOVES = {
    'U': -3,
    'D':  3,
    'L': -1,
    'R':  1
}

def is_valid_move(pos, move):
    """Check if blank can move in given direction."""
    if move == 'U' and pos < 3: return False
    if move == 'D' and pos > 5: return False
    if move == 'L' and pos % 3 == 0: return False
    if move == 'R' and pos % 3 == 2: return False
    return True

def neighbors(state):
    """Generate valid neighbor states from current state."""
    new_states = []
    pos = state.index(0)  # blank position
    for move, offset in MOVES.items():
        if is_valid_move(pos, move):
            new_pos = pos + offset
            new_state = list(state)
            new_state[pos], new_state[new_pos] = new_state[new_pos], new_state[pos]
            new_states.append((tuple(new_state), move))
```

```
    return new_states

def manhattan(state):
    """Manhattan distance heuristic."""
    distance = 0
    for idx, value in enumerate(state):
        if value == 0:
            continue
        goal_idx = GOAL.index(value)
        r1, c1 = divmod(idx, 3)
        r2, c2 = divmod(goal_idx, 3)
        distance += abs(r1 - r2) + abs(c1 - c2)
    return distance

def is_solvable(state):
    """Check if the 8-puzzle is solvable."""
    flat = [x for x in state if x != 0]
    inversions = 0
    for i in range(len(flat)):
        for j in range(i + 1, len(flat)):
            if flat[i] > flat[j]:
                inversions += 1
    return inversions % 2 == 0

def reconstruct_path(parent, moves, current):
    """Reconstruct path of moves."""
    path = []
    while current in parent:
        path.append(moves[current])
        current = parent[current]
    return path[::-1]

def a_star(start):
    """A* search algorithm for 8-puzzle."""
    if not is_solvable(start):
        return None

    open_list = []
    g = {start: 0}
    parent = {}
    moves = {}

    heapq.heappush(open_list, (manhattan(start), start))

    while open_list:
        f, current = heapq.heappop(open_list)

        if current == GOAL:
            return reconstruct_path(parent, moves, current)

        for neighbor, move in neighbors(current):
            tentative_g = g[current] + 1
            if neighbor not in g or tentative_g < g[neighbor]:
                g[neighbor] = tentative_g
                f_score = tentative_g + manhattan(neighbor)
                heapq.heappush(open_list, (f_score, neighbor))
                parent[neighbor] = current
```

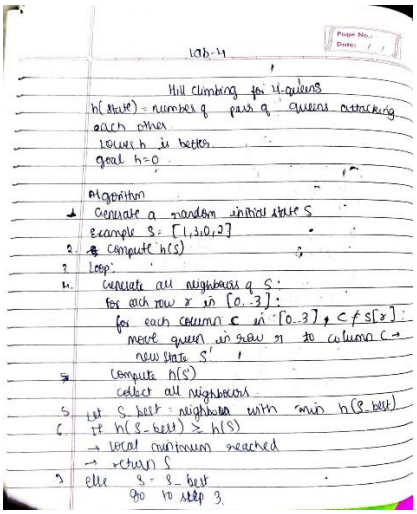
```
            moves[neighbor] = move
    return None

# --- MAIN PROGRAM ---
if __name__ == "__main__":
    print("Enter the start state of the 8-puzzle (use 0 for blank):")
    nums = []
    for i in range(9):
        nums.append(int(input(f"Tile {i+1}: ")))
    start = tuple(nums)

    solution = a_star(start)

    if solution:
        print(f"\nSolution found in {len(solution)} moves:")
        print(" -> ".join(solution))
    else:
        print("\nThis puzzle configuration is unsolvable!")
```

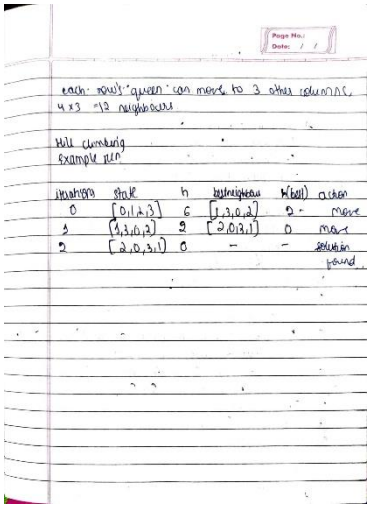
```
In [9]: runfile('C:/Users/BMSCECSE/.spyder-py3/untitled0.py',
Enter the start state of the 8-puzzle (use 0 for blank):
Tile 1: 1
Tile 2: 2
Tile 3: 3
Tile 4: 4
Tile 5: 0
Tile 6: 6
Tile 7: 7
Tile 8: 5
Tile 9: 8

Solution found in 2 moves:
```

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:





Code:

```
import random

def heuristic(board):
    h = 0
    n = len(board)
    for i in range(n):
        for j in range(i+1, n):
            if board[i] == board[j] or abs(board[i]-board[j]) == abs(i-j):
                h += 1
    return h

def hill_climbing_restart(initial_board, max_restarts=100):
    N = len(initial_board)
    board = [x-1 for x in initial_board]  # 0-based
    h = heuristic(board)

    restart_count = 0
    while h != 0 and restart_count < max_restarts:
        steps = 0
        while True:
            best_board = board[:]
            best_h = h
            for col in range(N):
                for row in range(N):
                    if row != board[col]:
                        neighbor = board[:]
                        neighbor[col] = row
                        h_neighbor = heuristic(neighbor)
                        if h_neighbor < best_h:
                            best_board = neighbor
                            best_h = h_neighbor
            steps += 1
            if best_h >= h:  # stuck
                break
            board = best_board
            h = best_h
            if h == 0:
                break
        if h == 0:
            print(f"Solution found after {restart_count} restarts and {steps} steps.")
            break
        # Random restart
        board = [random.randint(0, N-1) for _ in range(N)]
        h = heuristic(board)
        restart_count += 1

    return [x+1 for x in board], h

# User input
N = int(input("Enter number of queens (N): "))
print(f"Enter the initial positions of {N} queens (row numbers 1 to {N}):")
initial_board = list(map(int, input().split()))

solution, h_val = hill_climbing_restart(initial_board)
print("Final board:", solution)
```

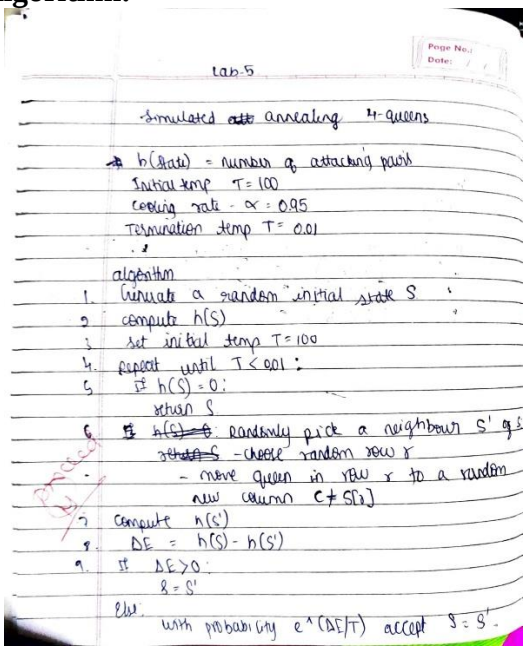
print("Heuristic H =", h_val)

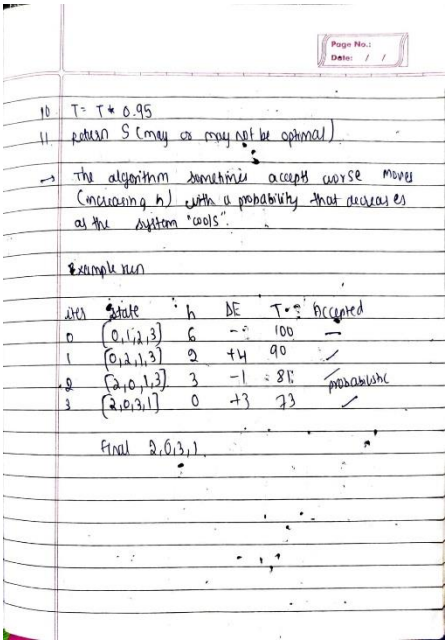
```
In [2]: runfile('C:/Users/student/untitled0.py', wdir='C:/Users/student')
Enter number of queens (N): 4
Enter the initial positions of 4 queens (row numbers 1 to 4):
1 2 1 4
Solution found after 4 restarts and 1 steps.
Final board: [2, 4, 1, 3]
Heuristic H = 0

In [3]: runfile('C:/Users/student/untitled0.py', wdir='C:/Users/student')
Enter number of queens (N): 4
Enter the initial positions of 4 queens (row numbers 1 to 4):
3 4 1 2
Solution found after 0 restarts and 3 steps.
Final board: [2, 4, 1, 3]
Heuristic H = 0
```

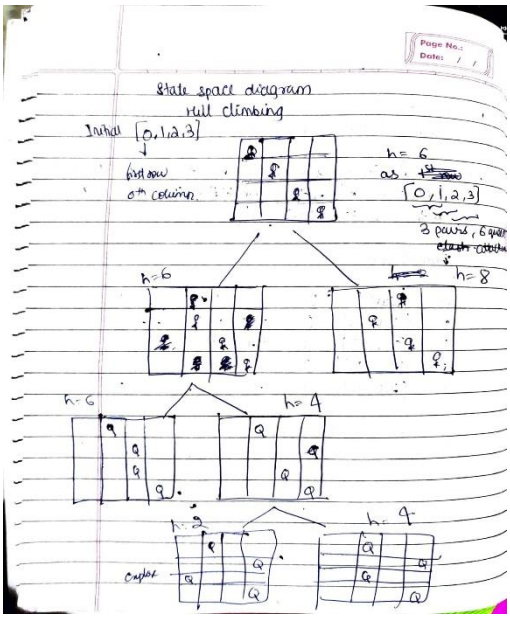
Program 5

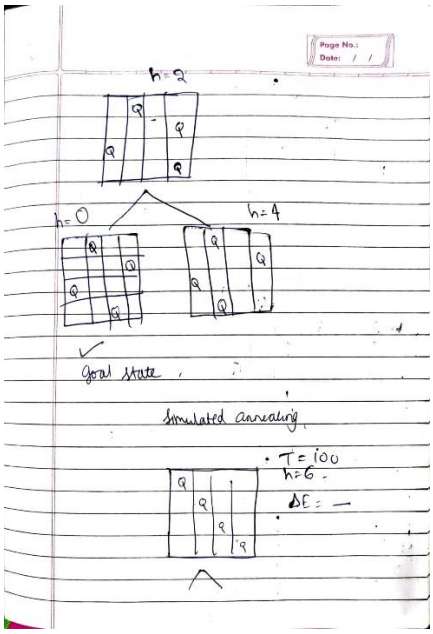
Simulated Annealing to Solve 8-Queens problem

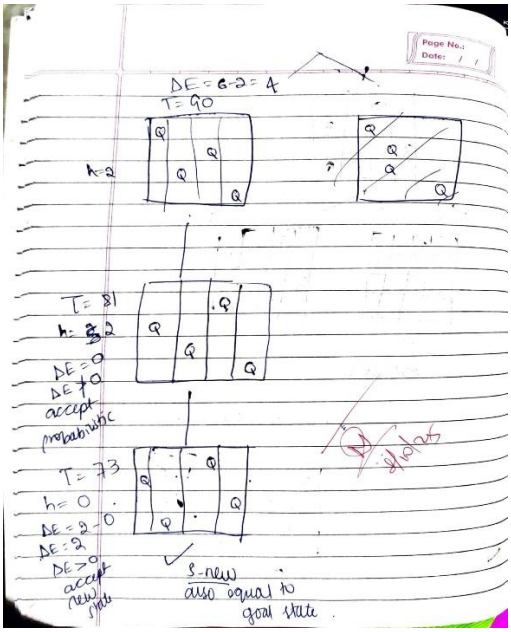
Algorithm:











Code:

```
import random
import math

def heuristic(board):
    """Count number of attacking pairs of queens."""
    h = 0
    n = len(board)
    for i in range(n):
        for j in range(i+1, n):
            if board[i] == board[j] or abs(board[i]-board[j]) == abs(i-j):
                h += 1
    return h

def simulated_annealing_user_initial(board, initial_temp, cooling_rate):
    """Simulated Annealing starting from user-provided initial board."""
```

```
    N = len(board)
    board = [x-1 for x in board]  # convert to 0-based index
    h = heuristic(board)
    T = initial_temp

    steps = 0
    while T > 0 and h != 0:
        col = random.randint(0, N-1)
        row = random.randint(0, N-1)
        while row == board[col]:
            row = random.randint(0, N-1)
        neighbor = board[:]
        neighbor[col] = row
        h_neighbor = heuristic(neighbor)
        delta_e = h_neighbor - h

        # Accept move
        if delta_e < 0:
            board = neighbor
            h = h_neighbor
        else:
            if random.random() < math.exp(-delta_e / T):
                board = neighbor
                h = h_neighbor

        T *= cooling_rate
        steps += 1

    if h == 0:
        print(f"Solution found in {steps} steps.")
    else:
        print(f"Stopped after {steps} steps. Final H = {h} (may be stuck).")

    return [x+1 for x in board], h  # convert back to 1-based indexing

# User input
N = int(input("Enter number of queens (N): "))
print(f"Enter the initial positions of {N} queens (row numbers 1 to {N}):")
initial_board = list(map(int, input().split()))
initial_temp = float(input("Enter initial temperature: "))
cooling_rate = float(input("Enter cooling rate (0 < rate < 1): "))

solution, h_val = simulated_annealing_user_initial(initial_board, initial_temp, cooling_rate)
print("Final board:", solution)
print("Heuristic H =", h_val)
```

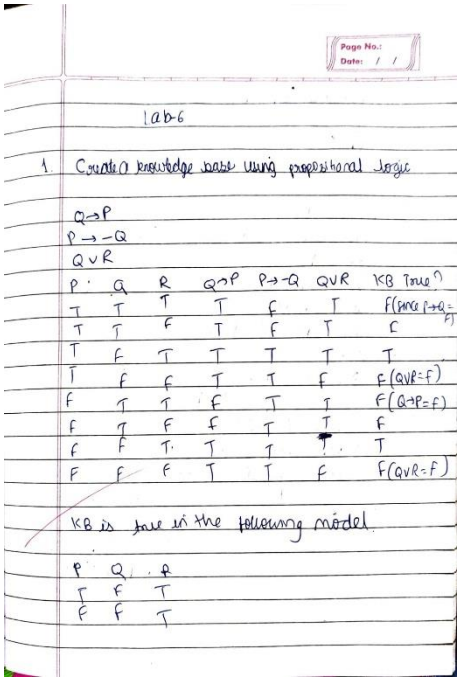
```
In [1]: runfile('C:/Users/student/untitled1.py', wdir='C:/Users/student')
Enter number of queens (N): 4
Enter the initial positions of 4 queens (row numbers 1 to 4):
1 2 1 4
Enter initial temperature: 10
Enter cooling rate (0 < rate < 1): .9
Solution found in 9 steps.
Final board: [3, 1, 4, 2]
Heuristic H = 0
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the

knowledge base or not.

Algorithm:



Lab-6

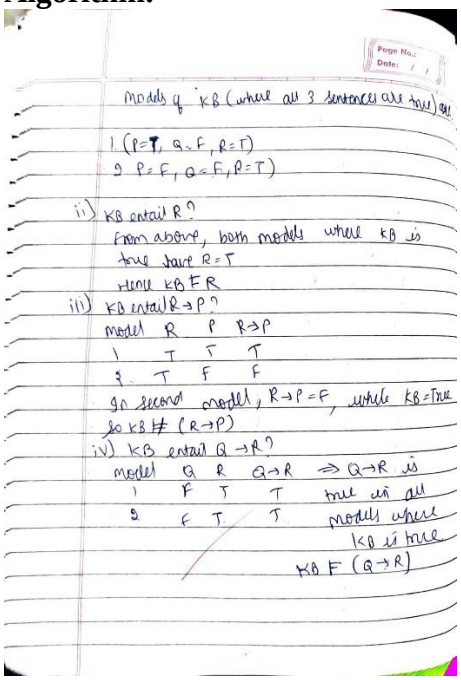
1. Create a knowledge base using propositional logic

$Q \rightarrow P$
$P \rightarrow \neg Q$
$Q \vee R$

P	Q	R	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$	KB True?
T	T	T	T	F	T	F (since $P \rightarrow Q$ = F)
T	T	F	T	F	T	F
T	F	T	T	T	T	T
T	F	F	T	T	F	F ($Q \vee R$ = F)
F	T	T	F	T	T	F ($Q \rightarrow P$ = F)
F	T	F	F	T	T	F
F	F	T	T	T	T	T
F	F	F	T	T	F	F ($Q \vee R$ = F)

KB is true in the following model.

P	Q	R
T	F	T
F	F	T



Models of KB (where all 3 sentences are true)

1. (P=T, Q=F, R=T)
2. P=F, Q=F, R=T)

ii) KB entail R?
From above, both models where KB is true have R=T
Hence KB ⊨ R

iii) KB entail $R \rightarrow P$?

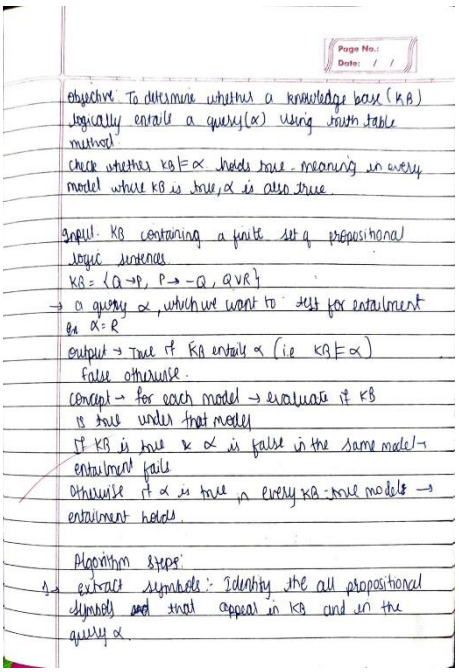
model	R	P	$R \rightarrow P$
1	T	T	T
2	T	F	F

In second model, $R \rightarrow P$ = F, while KB=True
So KB ⊭ $(R \rightarrow P)$

iv) KB entail $Q \rightarrow R$?

model	Q	R	$Q \rightarrow R$
1	F	T	T
2	F	F	T

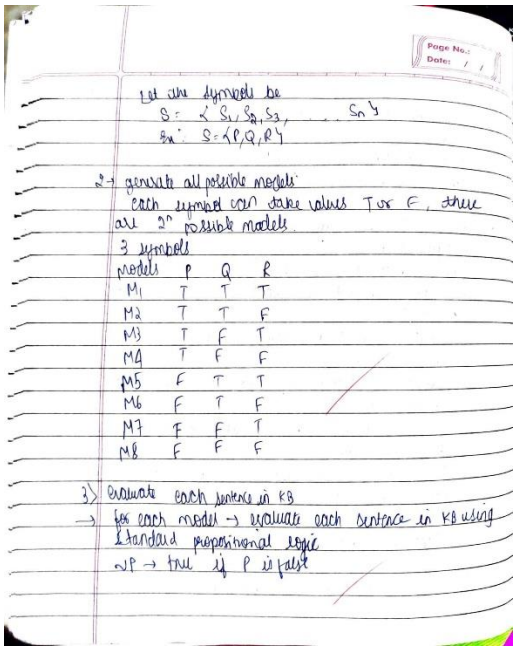
$\Rightarrow Q \rightarrow R$ is true in all models where KB is true
KB ⊨ $(Q \rightarrow R)$



Objective: To determine whether a knowledge base (KB) logically entails a query(α) using truth table method.
Check whether KB ⊨ α holds true - meaning in every model where KB is true, α is also true.

Input: KB containing a finite set of propositional logic sentences.
KB = {$Q \rightarrow P$, $P \rightarrow \neg Q$, $Q \vee R$}
→ a query α, which we want to test for entailment
Eg: $\alpha = R$
output → True if KB entails α (i.e KB ⊨ α)
False otherwise.
Concept → for each model → evaluate if KB is true under that model
If KB is true & α is false in the same model → entailment fails
Otherwise if α is true in every KB-true models → entailment holds.

Algorithm steps:
1. extract symbols: Identify the all propositional symbols that appear in KB and in the query α.

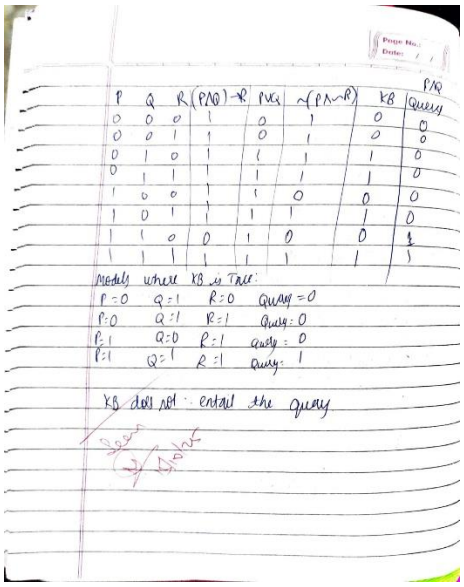


Let the symbols be
S = [$S_1, S_2, S_3, \ldots S_n$]
Eg: S = {P, Q, R}

2. generate all possible models
each symbol can take values T or F, there are 2^n possible models
3 symbols

models	P	Q	R
M_1	T	T	T
M_2	T	T	F
M_3	T	F	T
M_4	T	F	F
M_5	F	T	T
M_6	F	T	F
M_7	F	F	T
M_8	F	F	F

3) evaluate each sentence in KB
→ for each model → evaluate each sentence in KB using standard propositional logic
eg: P → true if P is false



P	Q	R	(P∧Q)→R	P∧Q	¬(P∧¬R)	KB	P/R Query
0	0	0	1	0	1	0	0
0	0	1	1	0	1	0	0
0	1	0	1	0	1	1	0
0	1	1	1	1	1	1	0
1	0	0	1	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	1	0	0	1
1	1	1	1	1	1	1	1

Models where KB is True:
P=0 Q=1 R=0 Query=0
P=0 Q=1 R=1 Query=0
P=1 Q=0 R=1 query=0
P=1 Q=1 R=1 Query=1

KB does not entail the query

Code:

PROPOSITIONAL LOGIC

```
from itertools import product

# Function for logical implication
def imply(a, b):
    return (not a) or b

# Convert user input to Python-evaluable expressions
import re

def convert_to_python_syntax(s):
    # Replace logical operators
    s = s.replace('!','not ')
    s = s.replace('∧',' and ')
    s = s.replace('∨',' or ')
    s = s.replace('↔','==')

    # Replace -> with imply(a,b) properly
    pattern = r'([A-Za-z() not]+)\s*->\s*([A-Za-z() not]+)'
    while re.search(pattern, s):
        s = re.sub(pattern, r'imply(\1, \2)', s)

    return s


# Input KB
n = int(input("Enter number of KB sentences: "))
kb_original = []
kb_converted = []

for i in range(n):
    s = input(f"Enter sentence {i+1} (use !, ∧, ∨, ->, ↔): ")
    kb_original.append(s)
    kb_converted.append(convert_to_python_syntax(s))
# Input Queries
queries_input = input("Enter queries separated by comma: ")
queries_original = [q.strip() for q in queries_input.split(',')]
```

```
queries_converted = [convert_to_python_syntax(q) for q in queries_original]

# Extract all variables from KB and queries
import re
variables = set()
for sentence in kb_original + queries_original:
    vars_in_s = re.findall(r'\b[A-Za-z]\b', sentence)
    variables.update(vars_in_s)
variables = sorted(list(variables))  # sort for consistent order

# Generate all possible truth assignments
assignments = list(product([False, True], repeat=len(variables)))

# Print Truth Table Header
header = variables + kb_original + ['KB True?']
print("\nTruth Table:")
print('  '.join(f"{h:<8}" for h in header))
print('-' * (len(header) * 10))

# Evaluate each row
models = []  # rows where KB is True
for vals in assignments:
    valuation = dict(zip(variables, vals))
    kb_values = [eval(expr, {"imply": imply}, valuation) for expr in kb_converted]
    kb_true = all(kb_values)
    if kb_true:
        models.append(valuation)
    row = vals + tuple(kb_values) + (kb_true,)
    print('  '.join(f"{str(x):<8}" for x in row))

# Entailment check
print("\nModels where KB is True:")
for m in models:
    print(m)

print("\nEntailment results:")
for i, query in enumerate(queries_converted):
    entails = all(eval(query, {"imply": imply}, m) for m in models)
    print(f"Does KB entail {queries_original[i]}? --> {entails}")
```

```
In [1]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
Enter number of KB sentences: 3
Enter sentence 1 (use !, ∧, ∨, ->, ⇔): Q->P
Enter sentence 2 (use !, ∧, ∨, ->, ⇔): P->!Q
Enter sentence 3 (use !, ∧, ∨, ->, ⇔): Q∨R
Enter queries separated by comma: R , R->P , Q->R

Truth Table:
P         Q         R         Q->P      P->!Q     Q∨R       KB True?
-------------------------------------------------------------------
False     False     False     True      True      False     False
False     False     True      True      True      True      True
False     True      False     False     True      True      False
False     True      True      False     True      True      False
True      False     False     True      True      False     False
True      False     True      True      True      True      True
True      True      False     True      False     True      False
True      True      True      True      False     True      False

Models where KB is True:
{'P': False, 'Q': False, 'R': True}
{'P': True, 'Q': False, 'R': True}

Entailment results:
Does KB entail R? --> True
Does KB entail R->P? --> False
Does KB entail Q->R? --> True
```

```
In [2]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
Enter number of KB sentences: 1
Enter sentence 1 (use !, ∧, ∨, ->, ⇔): (A∨C)∧(B∨!C)
Enter queries separated by comma: A∨B

Truth Table:
A         B         C         (A∨C)∧(B∨!C)  KB True?
-------------------------------------------------
False     False     False     False     False
False     False     True      False     False
False     True      False     False     False
False     True      True      True      True
True      False     False     True      True
True      False     True      False     False
True      True      False     True      True
True      True      True      True      True

Models where KB is True:
{'A': False, 'B': True, 'C': True}
{'A': True, 'B': False, 'C': False}
{'A': True, 'B': True, 'C': False}
{'A': True, 'B': True, 'C': True}

Entailment results:
Does KB entail A∨B? --> True
```

WUMPUS WORLD

```
import itertools
```

```
# Input KB
num_sentences = int(input("Enter number of KB sentences: "))
kb_sentences = []
all_vars = set()

for i in range(num_sentences):
    s = input(f"Enter sentence {i+1} (use !, ∧, ∨, ->, ↔): ")
    kb_sentences.append(s)
    # extract variable names (alphanumeric strings)
    tokens = s.replace("(", " ").replace(")", " ").replace("!", " ").replace("∧", " ").replace("∨", " ").replace("->", " ").replace("<->",
" ").split()
    for t in tokens:
        if t.isalnum() and not t.isdigit():
            all_vars.add(t)

# Input queries
queries_input = input("Enter queries separated by comma: ")
queries = [q.strip() for q in queries_input.split(",")]
for q in queries:
    all_vars.add(q)

# Prepare the KB expressions in Python syntax
kb_py = []
for s in kb_sentences:
    s_py = s
    s_py = s_py.replace("∧", " and ").replace("∨", " or ").replace("!", " not ").replace("->", "<=").replace("<->", "==")
    kb_py.append(s_py)

# Prepare queries in Python syntax
queries_py = []
for q in queries:
    q_py = q
    queries_py.append(q_py)

# Generate all truth assignments
var_list = list(all_vars)

# Print truth table header
header = "  ".join(var_list) + "  " + "  ".join([f"{s}" for s in kb_sentences]) + "  KB True?"
print("\nTruth Table:")
print(header)
print("-"*len(header.expandtabs()))

# Fill truth table
for values in itertools.product([False, True], repeat=len(var_list)):
    env = dict(zip(var_list, values))
    kb_values = [eval(expr, {}, env) for expr in kb_py]
    kb_true = all(kb_values)
    row = "  ".join([str(env[v]) for v in var_list]) + "  " + "  ".join([str(val) for val in kb_values]) + "  " + str(kb_true)
    print(row)

# Show models where KB is True
print("\nModels where KB is True:")
for values in itertools.product([False, True], repeat=len(var_list)):
    env = dict(zip(var_list, values))
    if all(eval(expr, {}, env) for expr in kb_py):
        print({k:v for k,v in env.items()})
```

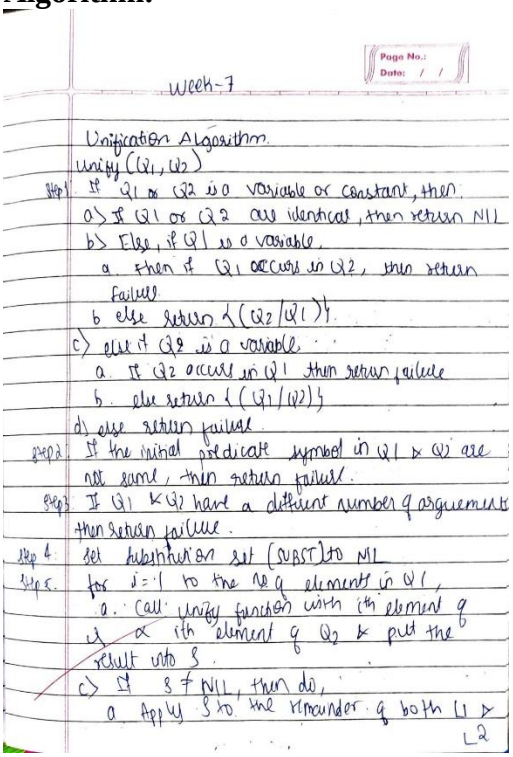
```
# Check entailment for queries
print("\nEntailment results:")
for q, q_py in zip(queries, queries_py):
    entails = all(not all(eval(expr, {}, dict(zip(var_list, vals))) for expr in kb_py) or eval(q_py, {}, dict(zip(var_list, vals))) for vals
in itertools.product([False, True], repeat=len(var_list)))
    print(f"Does KB entail {q}? --> {entails}")
```

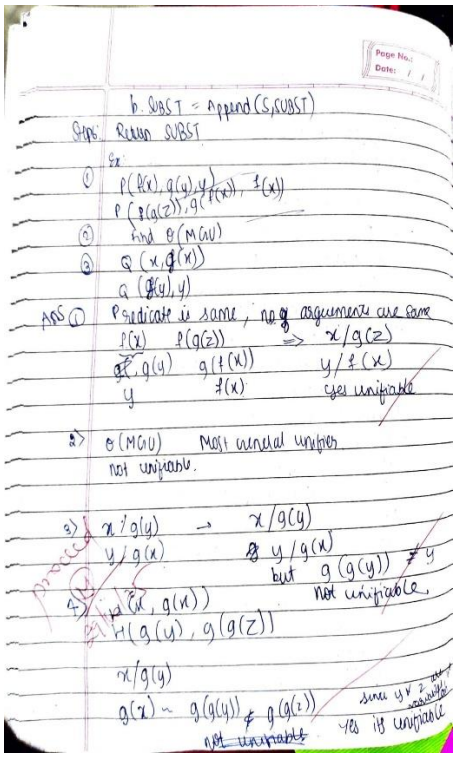
```
In [7]: runfile('C:/Users/student/untitled3.py', wdir='C:/Users/student')
Enter number of KB sentences: 2
Enter sentence 1 (use !, ∧, ∨, ->, <->): B11  <-> (P12 ∨ P21)
Enter sentence 2 (use !, ∧, ∨, ->, <->): S11 <-> (W12 ∨ W21)
Enter queries separated by comma: P12, W21

Truth Table:
S11  P12  B11  W21  W12  P21  B11  <-> (P12 ∨ P21)  S11 <-> (W12 ∨ W21)  KB True?
-------------------------------------------------------------------------------
False  False  False  False  False  False  True  True  True
False  False  False  False  False  True  False  True  False
False  False  False  False  True  False  True  False  False
False  False  False  False  True  True  False  False  False
False  False  False  True  False  False  True  False  False
False  False  False  True  False  True  False  False  False
False  False  False  True  True  False  True  False  False
False  False  False  True  True  True  False  False  False
False  False  True  False  False  False  False  True  False
False  False  True  False  False  True  True  True  True
False  False  True  False  True  False  False  False  False
False  False  True  False  True  True  False  False  False
False  False  True  True  False  False  False  False  False
False  False  True  True  False  True  True  False  False
False  False  True  True  True  False  False  False  False
False  False  True  True  True  True  True  False  False
False  True  False  False  False  False  False  True  False
False  True  False  False  False  True  False  True  False
False  True  False  False  True  False  False  False  False
False  True  False  False  True  True  False  False  False
False  True  False  True  False  False  False  False  False
False  True  False  True  False  True  False  False  False
False  True  False  True  True  False  False  False  False
False  True  False  True  True  True  False  False  False
False  True  True  False  False  False  True  True  True
False  True  True  False  False  True  True  True  True
False  True  True  False  True  False  True  False  False
False  True  True  False  True  True  True  False  False
False  True  True  True  False  False  True  False  False
False  True  True  True  False  True  True  False  False
False  True  True  True  True  False  True  False  False
False  True  True  True  True  True  False  False  False
True  False  False  False  False  False  True  False  False
True  False  False  False  False  True  False  False  False
```

Program 7

Implement unification in first order logic

Algorithm:





Code:

```
import re

def is_variable(x):
    """A variable starts with a lowercase letter and has no parentheses."""
    return isinstance(x, str) and x[0].islower() and '(' not in x and ')' not in x

def occur_check(var, expr, theta):
    """Check if variable occurs inside expr (prevents infinite recursion)."""
    if var == expr:
        return True
    elif isinstance(expr, list):
        return any(occur_check(var, sub, theta) for sub in expr)
    elif expr in theta:
        return occur_check(var, theta[expr], theta)
    return False

def substitute(expr, theta):
    """Apply substitution θ to expr recursively."""
    if isinstance(expr, list):
        return [substitute(e, theta) for e in expr]
    elif expr in theta:
        return substitute(theta[expr], theta)
    else:
        return expr

def unify(x, y, theta=None):
    """Main unification function."""
    if theta is None:
        theta = {}

    if theta == "FAIL":
        return "FAIL"

    # Apply substitutions
    x = substitute(x, theta)
    y = substitute(y, theta)

    if x == y:
        return theta
    elif is_variable(x):
        return unify_var(x, y, theta)
    elif is_variable(y):
        return unify_var(y, x, theta)
    elif isinstance(x, list) and isinstance(y, list):
        # First element is the predicate / function symbol
        if x[0] != y[0]:
            return "FAIL"  # predicate/function mismatch
        if len(x) != len(y):
            return "FAIL"  # different arity
        for xi, yi in zip(x[1:], y[1:]):
            theta = unify(xi, yi, theta)
            if theta == "FAIL":
                return "FAIL"
        return theta
    else:
```

```
        return "FAIL"

def unify_var(var, x, theta):
    """Unify variable var with x."""
    if var in theta:
        return unify(theta[var], x, theta)
    elif x in theta:
        return unify(var, theta[x], theta)
    elif occur_check(var, x, theta):
        return "FAIL"
    else:
        new_theta = theta.copy()
        new_theta[var] = x
        return new_theta

# ---------- Parsing Functions ----------

def parse_expression(expr_str):
    """
    Convert a FOL expression string into a nested list.
    Example: "Knows(John,Father(x))" -> ['Knows', 'John', ['Father', 'x']]
    """
    expr_str = expr_str.strip()
    if '(' not in expr_str:
        return expr_str

    match = re.match(r'(\w+)\((.*)\)', expr_str)
    if not match:
        return expr_str

    functor = match.group(1)
    args_str = match.group(2)

    # Split arguments, considering nested parentheses
    args = []
    depth = 0
    current = ''
    for ch in args_str:
        if ch == ',' and depth == 0:
            args.append(parse_expression(current.strip()))
            current = ''
        else:
            if ch == '(':
                depth += 1
            elif ch == ')':
                depth -= 1
            current += ch
    if current:
        args.append(parse_expression(current.strip()))

    return [functor] + args

# ---------- Main Program ----------

def main():
    print("==== First-Order Logic Unification ====")
    expr1 = input("Enter first expression (e.g. Knows(John,x)): ").strip()
```

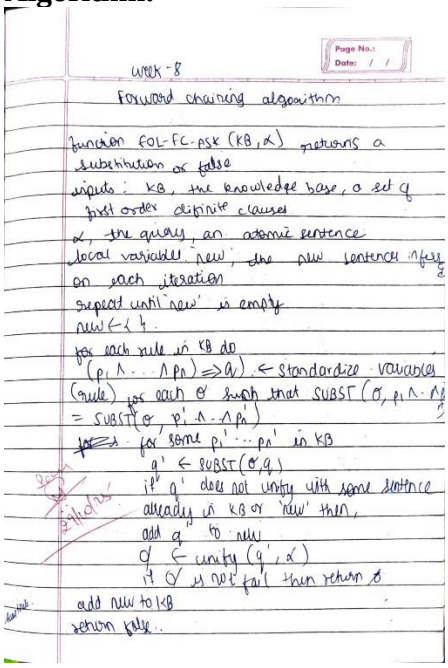
```
    expr2 = input("Enter second expression (e.g. Knows(y,Bill)): ").strip()

    x = parse_expression(expr1)
    y = parse_expression(expr2)

    result = unify(x, y)
    print("\n--- Result ---")
    if result == "FAIL":
        print("The expressions CANNOT be unified.")
    else:
        print("Unifier found:")
        for var, val in result.items():
            print(f"  {var} → {val}")

if __name__ == "__main__":
    main()
```

```
In [9]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
==== First-Order Logic Unification ====
Enter first expression (e.g. Knows(John,x)): brother(john, josh)
Enter second expression (e.g. Knows(y,Bill)): sister(jimmy, jam)

--- Result ---
❌ The expressions CANNOT be unified.

In [10]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
==== First-Order Logic Unification ====
Enter first expression (e.g. Knows(John,x)): brother(john, josh)
Enter second expression (e.g. Knows(y,Bill)): brother(josh , jun)

--- Result ---
✅ Unifier found:
  john → josh
  josh → jun

In [11]:
```

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:



Code:

```
import re

def isVariable(x):
    return len(x) == 1 and x.islower() and x.isalpha()

def getAttributes(string):
    expr = r'\([^)]+\)'
    matches = re.findall(expr, string)
    return matches

def getPredicates(string):
    expr = r'([a-zA-Z~]+)\([^&|]+\)'
    return re.findall(expr, string)

class Fact:
    def __init__(self, expression):
        self.expression = expression.strip()
        predicate, params = self.splitExpression(expression)
        self.predicate = predicate
        self.params = params

    def splitExpression(self, expression):
        predicate = getPredicates(expression)[0]
        params = getAttributes(expression)[0].strip('()').split(',')
        return [predicate, [p.strip() for p in params]]

    def substitute(self, var_map):
        params = [var_map.get(p, p) for p in self.params]
        return Fact(f"{self.predicate}({','.join(params)})")

    def __repr__(self):
        return self.expression

class Implication:
    def __init__(self, expression):
        self.expression = expression.strip()
        lhs, rhs = expression.split('=>')
        self.lhs = [Fact(f.strip()) for f in lhs.split('&')]
        self.rhs = Fact(rhs.strip())

    def infer(self, known_facts):
        substitutions = {}

        for fact in self.lhs:
            matched = False
            for known in known_facts:
                if known.predicate == fact.predicate:
                    mapping = {}
                    for i, param in enumerate(fact.params):
                        if isVariable(param):
                            mapping[param] = known.params[i]
                        elif param != known.params[i]:
                            break
                    else:
                        substitutions.update(mapping)
                        matched = True
```

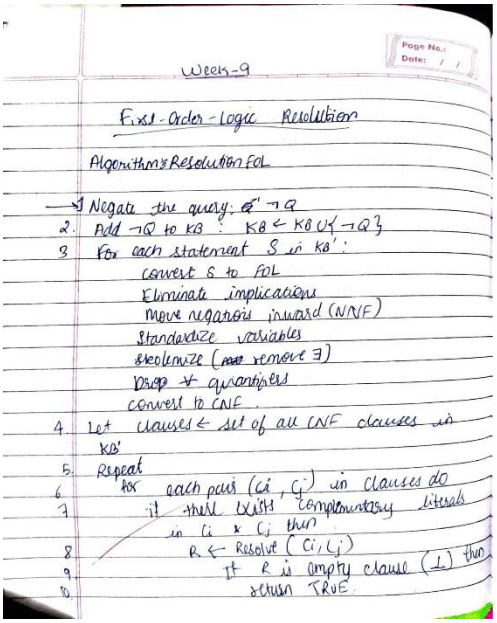
```
                break
        if not matched:
            return None

    return self.rhs.substitute(substitutions)

class KB:
    def __init__(self):
        self.facts = set()
        self.implications = set()

    def tell(self, expr):
        if '=>' in expr:
            self.implications.add(Implication(expr))
        else:
            self.facts.add(Fact(expr))

    def infer_all(self):
        added = True
        while added:
            added = False
            for rule in self.implications:
                new_fact = rule.infer(self.facts)
                if new_fact and new_fact.expression not in [f.expression for f in self.facts]:
                    print(f"Derived: {new_fact.expression}")
                    self.facts.add(new_fact)
                    added = True

    def ask(self, query):
        print(f"\nQuerying {query}:")
        self.infer_all()
        facts = [f.expression for f in self.facts]
        if query in facts:
            print(f"✅ Yes, {query.split('(')[1].strip(')')} is {query.split('(')[0]}.")
        else:
            print(f"❌ No, cannot infer {query}.")

    def display(self):
        print("\nAll facts in Knowledge Base:")
        for i, f in enumerate(sorted([f.expression for f in self.facts])):
            print(f"\t{i+1}. {f}")

def main():
    kb = KB()
    n = int(input("Enter number of FOL expressions: "))
    print("Enter expressions:")
    for _ in range(n):
        kb.tell(input().strip())

    query = input("Enter query: ").strip()
    kb.ask(query)
    kb.display()

if __name__ == "__main__":
    main()
```

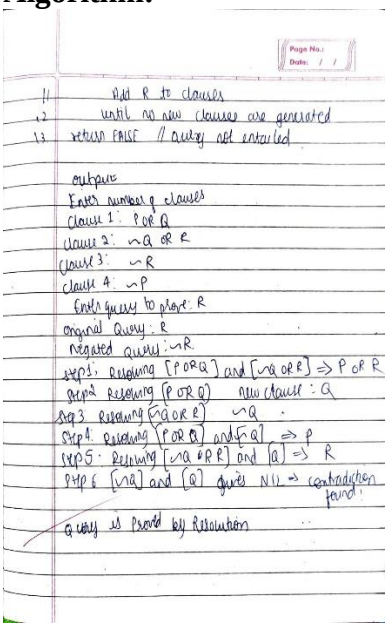
```
In [3]: runfile('C:/Users/student/untitled3.py', wdir='C:/Users/student')
Enter number of FOL expressions: 6
Enter expressions:
Man(Marcus)
Pompeian(Marcus)
Pompeian(x) => Roman(x)
Roman(x) => Loyal(x)
Man(x) => Person(x)
Person(x) => Mortal(x)
Enter query: Mortal(Marcus)

Querying Mortal(Marcus):
Derived: Person(Marcus)
Derived: Mortal(Marcus)
Derived: Roman(Marcus)
Derived: Loyal(Marcus)
 Yes, Marcus is Mortal.

All facts in Knowledge Base:
    1. Loyal(Marcus)
    2. Man(Marcus)
    3. Mortal(Marcus)
    4. Person(Marcus)
    5. Pompeian(Marcus)
    6. Roman(Marcus)
```

Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution

Algorithm:





Code:

```
def disjunctify(clauses):
    disjuncts = []
    for clause in clauses:
        disjuncts.append(tuple(clause.split('v')))
    return disjuncts

def getResolvant(ci, cj, di, dj):
    resolvant = list(ci) + list(cj)
    resolvant.remove(di)
    resolvant.remove(dj)
    return tuple(resolvant)
```

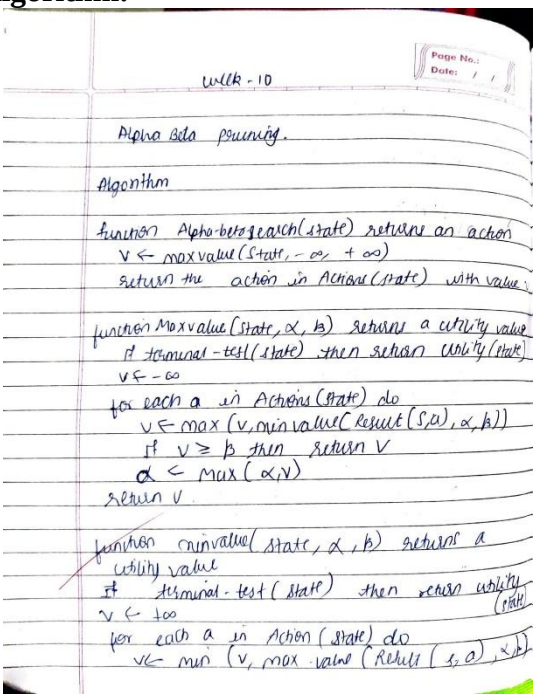
```
def resolve(ci, cj):
    for di in ci:
        for dj in cj:
            if di == '~' + dj or dj == '~' + di:
                return getResolvant(ci, cj, di, dj)

def checkResolution(clauses, query):
    clauses += [query if query.startswith('~') else '~' + query]

    proposition = '^'.join(['(' + clause + ')' for clause in clauses])
    print(f'Trying to prove {proposition} by contradiction....')

    clauses = disjunctify(clauses)
    resolved = False
    new = set()

    while not resolved:
        n = len(clauses)
        pairs = [(clauses[i], clauses[j]) for i in range(n) for j in range(i + 1, n)]

        for (ci, cj) in pairs:
            resolvant = resolve(ci, cj)
            if not resolvant:
                resolved = True
                break
            new = new.union(set([resolvant]))

        if new.issubset(set(clauses)):
            break

        for clause in new:
            if clause not in clauses:
                clauses.append(clause)

    if resolved:
        print('Knowledge Base entails the query, proved by resolution')
    else:
        print("Knowledge Base doesn't entail the query, no empty set produced after resolution")

clauses = input('Enter the clauses (separated by spaces): ').split()
query = input('Enter the query: ')
checkResolution(clauses, query)
```

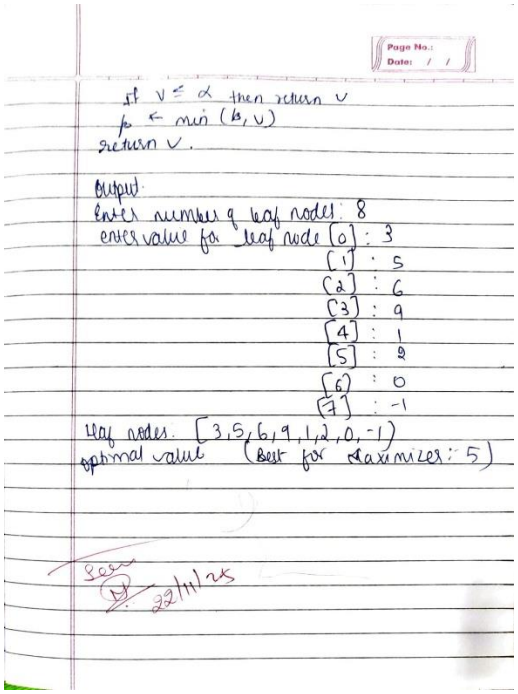
```
Enter the clauses (separated by spaces): ~AvB ~BvC A
Enter the query: C
Trying to prove (~AvB)^(~BvC)^(A)^(~C) by contradiction....
Knowledge Base entails the query, proved by resolution
```

Program 10

Implement Alpha-Beta Pruning.

Algorithm:





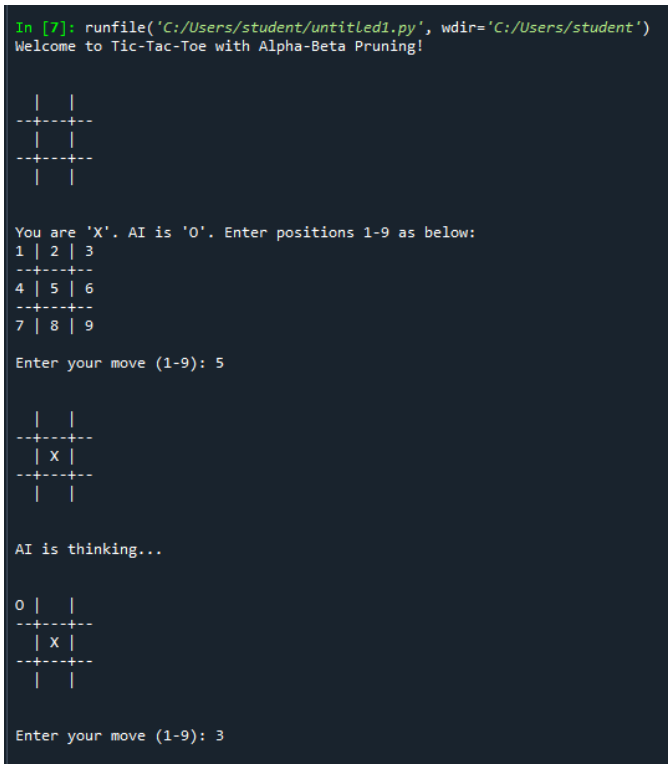
Code:

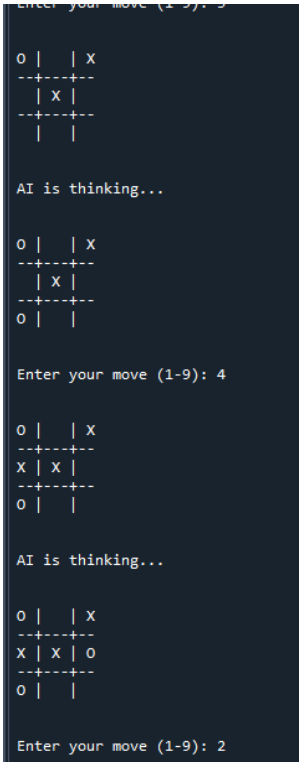
1. **Tic tac toe**

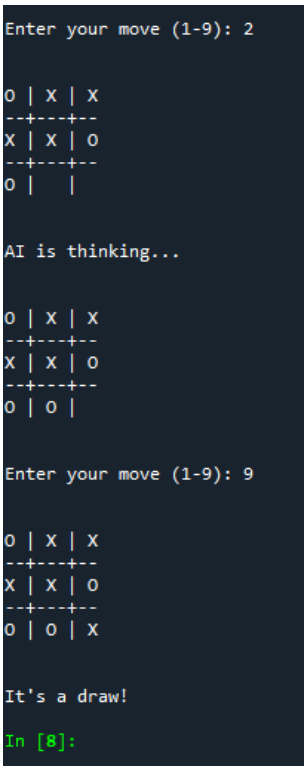
```
import math

# ---------- GAME LOGIC ----------

def print_board(board):
    print("\n")
    for i in range(3):
        print(" | ".join(board[i*3:(i+1)*3]))
        if i < 2:
            print("--+---+--")
    print("\n")

def check_winner(board):
    """Return 'X' or 'O' if there is a winner, or None otherwise."""
    win_combos = [
        [0,1,2], [3,4,5], [6,7,8],  # rows
        [0,3,6], [1,4,7], [2,5,8],  # columns
        [0,4,8], [2,4,6]            # diagonals
    ]
    for combo in win_combos:
        if board[combo[0]] == board[combo[1]] == board[combo[2]] and board[combo[0]] != ' ':
            return board[combo[0]]
    return None
```

```
def is_full(board):
    return ' ' not in board

# ----------- ALPHA-BETA PRUNING ------------

def minimax(board, depth, alpha, beta, maximizing):
    winner = check_winner(board)
    if winner == 'O':  # AI wins
        return 1
    elif winner == 'X':  # Human wins
        return -1
    elif is_full(board):  # Draw
        return 0

    if maximizing:
        max_eval = -math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                eval = minimax(board, depth + 1, alpha, beta, False)
                board[i] = ' '
                max_eval = max(max_eval, eval)
                alpha = max(alpha, eval)
                if beta <= alpha:
                    break
        return max_eval
    else:
        min_eval = math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                eval = minimax(board, depth + 1, alpha, beta, True)
                board[i] = ' '
                min_eval = min(min_eval, eval)
                beta = min(beta, eval)
                if beta <= alpha:
                    break
        return min_eval

def best_move(board):
    best_val = -math.inf
    move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            move_val = minimax(board, 0, -math.inf, math.inf, False)
            board[i] = ' '
            if move_val > best_val:
                best_val = move_val
                move = i
    return move

# ----------- GAME LOOP ------------

def play_game():
    board = [' ' for _ in range(9)]
```

```
    print("Welcome to Tic-Tac-Toe with Alpha-Beta Pruning!")
    print_board(board)
    print("You are 'X'. AI is 'O'. Enter positions 1-9 as below:")
    print("1 | 2 | 3\n--+---+--\n4 | 5 | 6\n--+---+--\n7 | 8 | 9\n")

    while True:
        # Human move
        while True:
            try:
                move = int(input("Enter your move (1-9): ")) - 1
                if move < 0 or move > 8 or board[move] != ' ':
                    print("Invalid move. Try again.")
                else:
                    board[move] = 'X'
                    break
            except ValueError:
                print("Please enter a valid number between 1 and 9.")

        print_board(board)
        if check_winner(board):
            print("You win! 🎉")
            break
        if is_full(board):
            print("It's a draw!")
            break

        # AI move
        print("AI is thinking...")
        ai_move = best_move(board)
        board[ai_move] = 'O'
        print_board(board)

        if check_winner(board):
            print("AI wins! 💻")
            break
        if is_full(board):
            print("It's a draw!")
            break

# ---------- RUN GAME ----------
if __name__ == "__main__":
    play_game()
```







2. 8 puzzle game

```
import heapq

goal_state = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]  # 0 represents the blank tile
]

# --- Helper functions ---

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i, j

def manhattan_distance(board):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = board[i][j]
            if value != 0:
                goal_i, goal_j = divmod(value - 1, 3)
                distance += abs(i - goal_i) + abs(j - goal_j)
    return distance

def get_neighbors(board):
    neighbors = []
    i, j = find_blank(board)
```

```
    moves = [(1,0), (-1,0), (0,1), (0,-1)]
    for di, dj in moves:
        new_i, new_j = i + di, j + dj
        if 0 <= new_i < 3 and 0 <= new_j < 3:
            new_board = [row[:] for row in board]
            new_board[i][j], new_board[new_i][new_j] = new_board[new_i][new_j], new_board[i][j]
            neighbors.append(new_board)
    return neighbors

def board_to_tuple(board):
    return tuple(tuple(row) for row in board)

def print_board(board):
    for row in board:
        print(" ".join(str(x) if x != 0 else '_' for x in row))
    print()

# --- A* Search Algorithm ---

def a_star(start):
    open_set = []
    heapq.heappush(open_set, (manhattan_distance(start), 0, start))
    came_from = {}
    g_score = {board_to_tuple(start): 0}
    visited = set()

    while open_set:
        _, cost, current = heapq.heappop(open_set)

        if current == goal_state:
            # Reconstruct path
            path = [current]
            while board_to_tuple(current) in came_from:
                current = came_from[board_to_tuple(current)]
                path.append(current)
            path.reverse()
            return path

        visited.add(board_to_tuple(current))

        for neighbor in get_neighbors(current):
            tentative_g = g_score[board_to_tuple(current)] + 1
            neighbor_tup = board_to_tuple(neighbor)

            if neighbor_tup in visited:
                continue

            if tentative_g < g_score.get(neighbor_tup, float('inf')):
                came_from[neighbor_tup] = current
                g_score[neighbor_tup] = tentative_g
                f_score = tentative_g + manhattan_distance(neighbor)
                heapq.heappush(open_set, (f_score, tentative_g, neighbor))

    return None  # No solution

# --- Main Program ---
```

```
if __name__ == "__main__":
    print("Enter initial 8-puzzle configuration (use 0 for blank):")
    start = []
    for i in range(3):
        row = list(map(int, input(f"Row {i+1}: ").split()))
        start.append(row)

    print("\nSolving...\n")
    solution_path = a_star(start)

    if solution_path:
        print(f"✅ Solution found in {len(solution_path)-1} steps.\n")
        for step, board in enumerate(solution_path):
            print(f"Step {step}:")
            print_board(board)
    else:
        print("❌ No solution exists for this puzzle.")
```

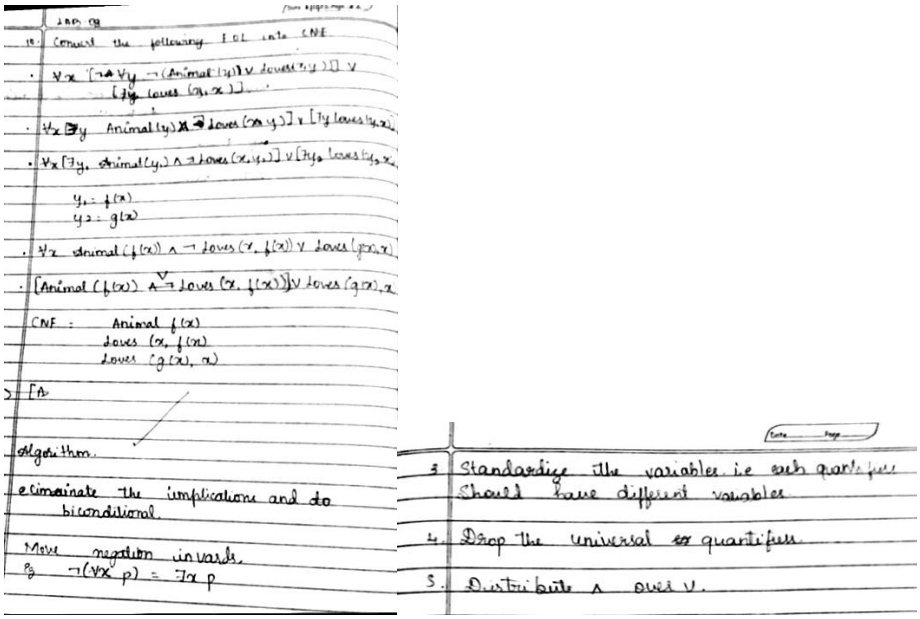
```
In [3]: runfile('C:/Users/student/untitled3.py', wdir='C:/Users/student')
Enter initial 8-puzzle configuration (use 0 for blank):
Row 1: 4 5 3
Row 2: 6 7 8
Row 3: 1 2 0

Solving...

✅ Solution found in 22 steps.

Step 0:
4 5 3
6 7 8
1 2 _

Step 1:
4 5 3
6 7 8
1 _ 2

Step 2:
4 5 3
6 _ 8
1 7 2

Step 3:
4 5 3
_ 6 8
```

```
Step 18:
1 2 3
5 _ 6
4 7 8

Step 19:
1 2 3
_ 5 6
4 7 8

Step 20:
1 2 3
4 5 6
_ 7 8

Step 21:
1 2 3
4 5 6
7 _ 8

Step 22:
1 2 3
4 5 6
7 8 _
```

Program 11

Convert a given first order logic statement into Conjunctive Normal Form (CNF)

Algorithm:



Code:

```
import re

def getAttributes(string):
    expr = r'\([^)]+\)'
    matches = re.findall(expr, string)
    return [m for m in str(matches) if m.isalpha()]

def getPredicates(string):
    expr = r'[A-Za-z~]+\([A-Za-z,]+\)'
    return re.findall(expr, string)

def DeMorgan(sentence):
    string = ''.join(list(sentence).copy())
    string = string.replace('~~', '')
    flag = '[' in string
    string = string.replace('~[', '')
    string = string.strip(']')
    for predicate in getPredicates(string):
        string = string.replace(predicate, f'~{predicate}')
    s = list(string)
    for i, c in enumerate(s):
        if c == 'V':
            s[i] = '^'
        elif c == '^':
            s[i] = 'V'
    string = ''.join(s)
    string = string.replace('~~', '')
    return f'[{string}]' if flag else string

def Skolemization(sentence):
    SKOLEM_CONSTANTS = [f'{chr(c)}' for c in range(ord('A'), ord('Z')+1)]
    statement = ''.join(list(sentence).copy())
    matches = re.findall('[∀∃].', statement)
    for match in matches[::-1]:
```

```
        statement = statement.replace(match, '')
    statements = re.findall(r'\[\[[^]]+\]\]', statement)
    for s in statements:
        statement = statement.replace(s, s[1:-1])
    for predicate in getPredicates(statement):
        attributes = getAttributes(predicate)
        if ''.join(attributes).islower():
            statement = statement.replace(predicate, predicate)
        else:
            aL = [a for a in attributes if a.islower()]
            aU = [a for a in attributes if not a.islower()][0] if attributes else ''
            if aU:
                statement = statement.replace(aU, f'{SKOLEM_CONSTANTS.pop(0)}({aL[0] if len(aL) else match[1]})')
    return statement

def clean_output(expr):
    # Remove multiple brackets and redundant negations
    expr = expr.replace('~~', '')
    while '[[' in expr or ']]' in expr:
        expr = expr.replace('[[', '[').replace(']]', ']')
    expr = expr.strip('[] ')
    # Remove redundant outer brackets like [(p | q)] -> p | q
    if expr.startswith('(') and expr.endswith(')'):
        expr = expr[1:-1]
    # Replace internal redundant patterns
    expr = re.sub(r'\s+', ' ', expr)
    return expr

def fol_to_cnf(fol):
    statement = fol.replace("<=>", "_")
    while '_' in statement:
        i = statement.index('_')
        new_statement = '[' + statement[:i] + '=>' + statement[i+1:] + ']^[' + statement[i+1:] + '=>' + statement[:i] + ']'
        statement = new_statement
    statement = statement.replace("=>", "-")

    expr = r'\[([^]]+)\]'
    statements = re.findall(expr, statement)
    for i, s in enumerate(statements):
        if '[' in s and ']' not in s:
            statements[i] += ']'
    for s in statements:
        statement = statement.replace(s, fol_to_cnf(s))

    while '-' in statement:
        i = statement.index('-')
        br = statement.index('[') if '[' in statement else 0
        new_statement = '~' + statement[br:i] + 'V' + statement[i+1:]
        statement = statement[:br] + new_statement if br > 0 else new_statement

    while '~∀' in statement:
        i = statement.index('~∀')
        statement = list(statement)
        statement[i], statement[i+1], statement[i+2] = '∃', statement[i+2], '~'
        statement = ''.join(statement)

    while '~∃' in statement:
```

```
            i = statement.index('~∃')
            s = list(statement)
            s[i], s[i+1], s[i+2] = '∀', s[i+2], '~'
            statement = ''.join(s)

        statement = statement.replace('~[∀', '[~∀')
        statement = statement.replace('~[∃', '[~∃')

        expr = r'(~[∀V∃].)'
        statements = re.findall(expr, statement)
        for s in statements:
            statement = statement.replace(s, fol_to_cnf(s))

        expr = r'~\[[^]]+\]'
        statements = re.findall(expr, statement)
        for s in statements:
            statement = statement.replace(s, DeMorgan(s))

        return statement

    def main():
        print("=== FOL to CNF Converter (Simplified Output) ===")
        print("Supports ∀, ∃, ~, &, |, >>, <=>, brackets [] () {}")
        print("Example 1: ∀x[~∀y~(Animal(y) | Loves(x,y)) | ∃y Loves(y,x)]")
        print("Example 2: ~(p >> q) | (r & s)")
        print("-----------------------------------------------------------")

        fol = input("Enter FOL formula: ").strip()
        try:
            result = Skolemization(fol_to_cnf(fol))
            print("\nSimplified CNF form:")
            print(clean_output(result))
        except Exception as e:
            print("\nError: Could not parse the formula.")
            print("Details:", e)

    if __name__ == "__main__":
        main()
```

```
In [7]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
=== FOL to CNF Converter (Simplified Output) ===
Supports ∀, ∃, ~, &, |, >>, <=>, brackets [] () {}
Example 1: ∀x[~∀y~(Animal(y) | Loves(x,y)) | ∃y Loves(y,x)]
Example 2: ~(p >> q) | (r & s)
------------------------------------------------------------
Enter FOL formula: ∀x[~∀y~(Animal(y) | Loves(x,y)) | ∃y Loves(y,x)]

Simplified CNF form:
Animal(y) | Loves(x,y)) | Loves(y,x

In [8]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
=== FOL to CNF Converter (Simplified Output) ===
Supports ∀, ∃, ~, &, |, >>, <=>, brackets [] () {}
Example 1: ∀x[~∀y~(Animal(y) | Loves(x,y)) | ∃y Loves(y,x)]
Example 2: ~(p >> q) | (r & s)
------------------------------------------------------------
Enter FOL formula: ~(p >> q) | (r & s)

Simplified CNF form:
~(p >> q) | (r & s)

In [9]: runfile('C:/Users/student/untitled2.py', wdir='C:/Users/student')
=== FOL to CNF Converter (Simplified Output) ===
Supports ∀, ∃, ~, &, |, >>, <=>, brackets [] () {}
Example 1: ∀x[~∀y~(Animal(y) | Loves(x,y)) | ∃y Loves(y,x)]
Example 2: ~(p >> q) | (r & s)
------------------------------------------------------------
Enter FOL formula: ~(∀x)(∃y)[~p(x) >> ~q(x,y)] & (∃x)(∀y)q(x,y)

Simplified CNF form:
~()()[~p(x) >> ~q(x,y)] & ()()q(x,y)
```